

Practice Set: Constructors in C++

Section 1: Conceptual & Practical Questions (1–10)

Question 1: Define what a **Constructor** is in C++ and explain its primary task during object instantiation. Why does relying on separate manual setter methods (e.g., setData()) lead to potential data corruption bugs?

Question 2: Explain the five **Core Characteristics** of constructors in C++ (Name Identity, Return Type, Invocation Timing, Static Restriction, and Access Visibility).

Question 3: Detail how constructors work in RAM during object creation. Trace the execution and RAM initialization flow when Complex c1(3, 4); is executed, referencing stack reservation and the implicit this pointer.

Question 4: Describe the three primary types of constructors supported in C++ based on parameter signatures: **Default Constructor**, **Parameterized Constructor**, and **Copy Constructor**.

Question 5: Outline the **Compiler Constructor Synthesis Rules** across the four scenarios: zero user-defined constructors, user defines a default constructor, user defines a parameterized constructor, and user defines a copy constructor.

Question 6: Explain the "**Missing Default Constructor Trap**". What compilation error occurs if a developer declares an object as Complex c2; after defining an explicit parameterized constructor, and how is it resolved?

Question 7: Explain the syntax varieties for constructor invocation, detailing the difference between **Functional (Implicit) Notation** and **Assignment (Explicit) Notation** for default, parameterized, and copy constructors.

Question 8: What is the "**Most Vexing Parse**" warning in C++? Why does writing Complex c1(); fail to invoke the default constructor, and how does the compiler interpret it instead?

Question 9: Why is **Pass-by-Reference** (`const ClassName &source`) strictly mandatory in a Copy Constructor declaration? Explain what happens to the Call Stack if the parameter is passed by value (e.g., `Complex(Complex c)`).

Question 10: Write a complete C++ program that implements a Complex class with:

A default constructor setting values to zero.

A two-argument parameterized constructor.

A single-argument constructor allowing implicit conversion.

An explicit copy constructor. Demonstrate object creation for each constructor type in `main()`.

Section 2: Interview & Viva Questions (11–20)

Question 11 (Viva): Can a constructor in C++ have a return type like `void` or `int`?

Question 12 (Viva): Can a constructor be declared with the `static` keyword? Why or why not?

Question 13 (Viva): Under what specific condition does the C++ compiler automatically synthesize a default constructor for a class?

Question 14 (Viva): Does the compiler generate an automatic default constructor if a user has explicitly written only a copy constructor?

Question 15 (Viva): How does the compiler interpret the line `Complex c1();` inside a function body?

Question 16 (Viva): What is the syntax for invoking a single-argument parameterized constructor using assignment notation?

Question 17 (Viva): What type of recursion and runtime failure occurs if a copy constructor accepts its argument by value?

Question 18 (Viva): Why are constructors generally placed inside the public section of a class declaration?

Question 19 (Viva): Through which hidden pointer does a constructor access and initialize an object's specific memory address during construction?

Question 20 (Viva): What initial values do primitive member variables hold in RAM before the constructor body executes?

Answers & Explanations

Answer: 01

Definition: A Constructor is a special member function of a class whose primary responsibility is to initialize an object's data members upon instantiation, ensuring it begins life in a valid state.

Setter Flaw: Relying on separate setter functions (e.g., `c1.setData(3, 4);`) depends on manual programmer discipline. If a developer forgets to call the setter, the object's primitive variables retain indeterminate bit patterns (garbage values), leading to corrupted program computations. Constructors eliminate this by coupling initialization directly with memory allocation.

Answer: 02

Identical Name: The constructor must share the exact same name as the class.

No Return Type: It has no return type (not even void) and cannot return values.

Automatic Invocation: It is invoked implicitly by the compiler at the instant an object is instantiated.

Cannot Be Static: It cannot be declared static because it requires an object context via the implicit `this` pointer.

Must Reside in Public: It is typically placed in the public section to permit instantiation by external scopes (like `main()`).

Answer: 03

When `Complex c1(3, 4);` executes:

Step 1 (Stack Reservation): The OS reserves 8 bytes of stack memory for two integers (`a` and `b`), which initially contain garbage bits.

Step 2 (Implicit Invocation): The compiler passes the starting address (e.g., 0x7ffd10) as the implicit this pointer to `Complex::Complex(int x, int y)` along with values 3 and 4.

Step 3 (Memory Initialization): The constructor executes, writing 3 to a (address 0x7ffd10) and 4 to b (address 0x7ffd14), placing the object into a guaranteed valid state before subsequent statements execute.

Answer: 04

Default Constructor: Accepts no arguments (or has all parameters defaulted) and initializes objects created without argument lists (e.g., `Complex c1;`).

Parameterized Constructor: Accepts one or more arguments to initialize an object with custom values at creation time (e.g., `Complex c1(3, 4);`).

Copy Constructor: Initializes a new object by copying the member variables from an already existing object of the same class (e.g., `Complex c2(c1);`).

Answer: 05

Zero constructors defined: Compiler generates both a **Default Constructor** and a **Copy Constructor**.

User defines a Default Constructor: Compiler generates a **Copy Constructor only**.

User defines a Parameterized Constructor: Compiler generates a **Copy Constructor only** (no default constructor is generated).

User defines a Copy Constructor: Compiler generates **nothing**.

Answer: 06

The Trap: When any parameterized constructor is defined by the developer, the compiler automatically revokes its synthesis of the default parameterless constructor.

Compilation Error: Attempting parameterless instantiation like `Complex c2;` results in: "no matching default constructor available".

Resolution: The developer must explicitly define an argument-free default constructor (e.g., `Complex() { a = 0; b = 0; }`) inside the class.

Answer: 07

Variety	Functional (Implicit) Notation	Assignment (Explicit) Notation
Default Constructor	Complex c1; (never Complex c1();)	Complex c1 = Complex();
Parameterized (2 Args)	Complex c1(3, 4);	Complex c1 = Complex(3, 4);
Single-Arg Conversion	Complex c2(5);	Complex c2 = 5;
Copy Constructor	Complex c4(c1);	Complex c4 = c1;

Answer: 08

Most Vexing Parse: A syntactic ambiguity in C++ where a statement intended to be an object definition is parsed by the compiler as a function declaration.

Mechanism: Writing `Complex c1();` does not invoke the default constructor; the compiler interprets it as a function prototype declaration for a function named `c1` that takes zero parameters and returns a `Complex` object. Parameterless objects must be declared simply as `Complex c1;`.

Answer: 09

Why Mandatory: In `ClassName(ClassName source)`, the parameter `source` is passed by value. Passing an object by value requires making an identical copy of it, which invokes the copy constructor again.

Stack Crash: This triggers an infinite recursive chain of copy constructor calls, causing memory exhaustion and crashing the program with a stack overflow. Using reference passing (`const ClassName &source`) directly aliases the original object without creating a new copy.

Answer: 10

```
#include <iostream>
using namespace std;
class Complex {private:
    int a, b;public:
    // 1. Default Constructor
    Complex()
    {
        a = 0;
```

```

        b = 0;
        cout << "Default constructor invoked!" << endl;
    }

    // 2. Parameterized Constructor (2 arguments)
    Complex(int x, int y)
    {
        a = x;
        b = y;
        cout << "Parameterized constructor (2 args) invoked!" << endl;
    }

    // 3. Single-Argument Constructor (implicit conversion)
    Complex(int k)
    {
        a = k;
        b = 0;
        cout << "Parameterized constructor (1 arg) invoked!" << endl;
    }

    // 4. Explicit Copy Constructor
    Complex(const Complex &c)
    {
        a = c.a;
        b = c.b;
        cout << "Copy constructor invoked successfully!" << endl;
    }

    void showData()
    {
        cout << "a = " << a << " | b = " << b << endl;
    }
};

int main()
{
    Complex c1;    // Default constructor
    c1.showData();

    Complex c2(3, 4); // Parameterized (2 args)
    c2.showData();

    Complex c3 = 5;   // Parameterized (1 arg conversion)
    c3.showData();

    Complex c4(c2);   // Copy constructor (functional)
    c4.showData();
}

```

```
Complex c5 = c2;    // Copy constructor (assignment)
c5.showData();

return 0;
}
```

Answer: 11

No; constructors have no return type whatsoever (not even void), and returning values from them is illegal.

Answer: 12

No; a constructor cannot be static because it must operate on a specific instance allocated in memory using the implicit this pointer, whereas static members lack object context.

Answer: 13

Only when the class definition contains **zero user-defined constructors** of any kind.

Answer: 14

No; if a user explicitly defines a copy constructor, the compiler generates nothing else.

Answer: 15

It interprets it as a function declaration named c1 that accepts zero arguments and returns a Complex object.

Answer: 16

Using the assignment conversion syntax: ClassName obj = value; (e.g., Complex c2 = 5;).

Answer: 17

It causes infinite recursive calls to the copy constructor, which exhausts stack memory and results in a stack overflow crash.

Answer: 18

To allow outside functions (like main() or factory functions) to freely instantiate objects of that class.

Answer: 19

The implicit **this pointer**, which passes the starting RAM address of the newly allocated object memory block.

Answer: 20

They contain uninitialized, indeterminate bit patterns commonly called **garbage values**.